



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS
PROFESIONALES
ZONA MEDIA

INGENIERÍA MECATRÓNICA



PRÁCTICA 8

**Contador Ascendente con Reinicio
usando Display de 7 Segmentos**

MATERIA: MICROCONTROLADORES

DOCENTE: Ing. JESÚS PADRÓN

ALUMNO: CARLOS ANGEL GARCÍA SIFUENTES

CLAVE: A383917

CARRERA: INGENIERÍA MECATRÓNICA

SEMESTRE: 4to SEMESTRE

FECHA: 20 DE MAYO DE 2026

Contador Hexadecimal Ascendente con Reinicio por Pulsador

Mediante Display de 7 Segmentos y Máscaras de Bits en RPi Pico W

CARLOS ÁNGEL GARCÍA SIFUENTES
a383917@alumnos.uaslp.mx

Fecha: 20 de mayo de 2026

Resumen—Se presenta el desarrollo de la Práctica 8, en la que se implementa un contador hexadecimal ascendente (0 a F) sobre un display de 7 segmentos de cátodo común, controlado desde la Raspberry Pi Pico W mediante programación en lenguaje C con el SDK oficial de Raspberry Pi. El programa emplea una tabla de patrones de bits (*lookup table*) para codificar cada dígito hexadecimal, y utiliza las funciones `gpio_set_mask()` y `gpio_clr_mask()` para escribir o borrar simultáneamente los 7 segmentos del display con una sola instrucción. Un pulsador configurado con resistencia *pull-up* interna permite reiniciar el contador en cualquier momento. El circuito avanza automáticamente un dígito por segundo y regresa a 0 al llegar a F.

I. INTRODUCCIÓN

I-A Descripción General de la Práctica

En esta práctica se integran dos conceptos fundamentales nuevos respecto a las prácticas anteriores: el manejo de un periférico de salida compuesto (el display de 7 segmentos) y el uso de máscaras de bits para controlar múltiples pines GPIO de forma simultánea y eficiente. El contador avanza automáticamente cada 1s, mostrando en el display los dígitos del 0 al F en hexadecimal, y puede ser reiniciado en cualquier momento presionando el pulsador.

I-B Trabajo Previo: Display de 7 Segmentos de Cátodo Común

El display de 7 segmentos es un dispositivo de visualización compuesto por siete LEDs individuales organizados en forma de dígito numérico, etiquetados con las letras A–G. En la configuración de **cátodo común**, todos los cátodos de los LEDs internos están unidos y conectados a GND; para encender un segmento basta con aplicar un nivel alto (3.3V) en el ánodo correspondiente. La Figura 1 muestra la distribución de segmentos y su mapeo a los pines GPIO.

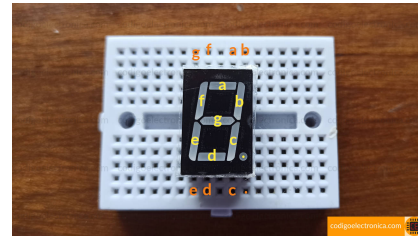


Figura 1: Distribución de segmentos A–G en un display de 7 segmentos de cátodo común.

La Tabla I muestra la asignación de cada segmento al pin GPIO correspondiente, siendo GPIO 2 el segmento A (bit 0 del patrón) hasta GPIO 8 el segmento G (bit 6 del patrón).

Tabla I: Asignación de segmentos a pines GPIO

Segmento	Bit del patrón	Pin GPIO
A	bit 0	GPIO 2
B	bit 1	GPIO 3
C	bit 2	GPIO 4
D	bit 3	GPIO 5
E	bit 4	GPIO 6
F	bit 5	GPIO 7
G	bit 6	GPIO 8

II. DESARROLLO

II-A Material y Recursos Utilizados

- **Hardware:** 1 Placa Raspberry Pi Pico W, 1 cable micro USB tipo B, 1 display de 7 segmentos de cátodo común, 1 resistencia de 220 Ω , 1 pulsador sin enclavamiento mecánico, cable UTP y 1 protoboard.
- **Software y librerías:** SDK de Raspberry Pi Pico, Pico – Visual Studio Code, librería `pico_stdlib` y cabecera `hardware/gpio.h`.

II-B Estructura del Proyecto y Configuración de CMake

Se configuró la carpeta `PRACTICA_8` con los archivos base del SDK. El archivo `CMakeLists.txt` utilizado es

el siguiente:

ARCHIVO CMakeLists.txt

```
cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)
set(PICO_BOARD pico_w)

project(PRACTICA_8)

pico_sdk_init()

add_executable(Reiniciador
    Reiniciador.c
)

target_link_libraries(Reiniciador
    pico_stdlib
)

pico_add_extra_outputs(Reiniciador)
```

A diferencia de prácticas anteriores, este proyecto incluye la cabecera `hardware/gpio.h` directamente en el código C, que es parte del SDK y se enlaza automáticamente a través de `pico_stdlib`.

II-C Implementación del Código en C

El archivo `Reiniciador.c` centraliza toda la lógica del contador en una estructura compacta. Sus componentes principales se describen a continuación.

II-C1. Tabla de patrones hexadecimales (lookup table)

El arreglo `hexDigits[16]` almacena un patrón de 7 bits por cada dígito hexadecimal (0-F). Cada bit del patrón corresponde a un segmento del display: el bit 0 activa el segmento A, el bit 1 activa el segmento B, y así sucesivamente hasta el bit 6 para el segmento G.

II-C2. Control de pines con `gpio_set_mask()` y `gpio_clr_mask()`

En lugar de llamar a `gpio_put()` siete veces (una por segmento), el programa desplaza el patrón del dígito actual `FIRST_GPIO` posiciones a la izquierda para alinearlos con los pines `GPIO2-8`, y luego aplica la máscara resultante en una sola llamada a `gpio_set_mask()`. Al terminar el segundo de visualización, `gpio_clr_mask()` apaga todos los segmentos antes de mostrar el siguiente dígito.

CÓDIGO FUENTE – Reiniciador.c

```
#include <stdio.h>
```

```
#include "pico/stdlib.h"
#include "hardware/gpio.h"

#define FIRST_GPIO 2 // Primer pin del
                    // display (segmento A)
#define BUTTON_GPIO 9 // Boton de
                    // reinicio

// Tabla de patrones para digitos 0-F (
// catodo comun, corregido ->)
int hexDigits[16] = {
    0x3F, // 0 -> 0111111
    0x06, // 1 -> 0000110
    0x5B, // 2 -> 1011011
    0x4F, // 3 -> 1001111
    0x66, // 4 -> 1100110
    0x6D, // 5 -> 1101101
    0x7D, // 6 -> 1111101
    0x07, // 7 -> 0000111
    0x7F, // 8 -> 1111111
    0x6F, // 9 -> 1101111
    0x77, // A -> 1110111
    0x7C, // B -> 1111100
    0x39, // C -> 0111001
    0x5E, // D -> 1011110
    0x79, // E -> 1111001
    0x71, // F -> 1110001
};

int main() {
    stdio_init_all();

    // Inicializar GPIO 2-8 como salidas (7
    // segmentos)
    for (int i = FIRST_GPIO; i < FIRST_GPIO
        + 7; i++) {
        gpio_init(i);
        gpio_set_dir(i, GPIO_OUT);
    }

    // Boton como entrada con pull-up
    // interno
    gpio_init(BUTTON_GPIO);
    gpio_set_dir(BUTTON_GPIO, GPIO_IN);
    gpio_pull_up(BUTTON_GPIO);

    int count = 0;

    while (true) {
        // Reiniciar si se presiona el
        // boton
        if (gpio_get(BUTTON_GPIO) == 0) {
            count = 0;
        }

        // Construir y aplicar mascara de
        // bits al display
        int32_t mask = hexDigits[count] <<
            FIRST_GPIO;
        gpio_set_mask(mask);
        sleep_ms(1000);
    }
}
```

```

gpio_clr_mask(mask); // Apagar
antes del siguiente dígito

// Incremento ciclico 0-15
count = (count + 1) % 16;
}
}

```

II-D Código cargado correctamente

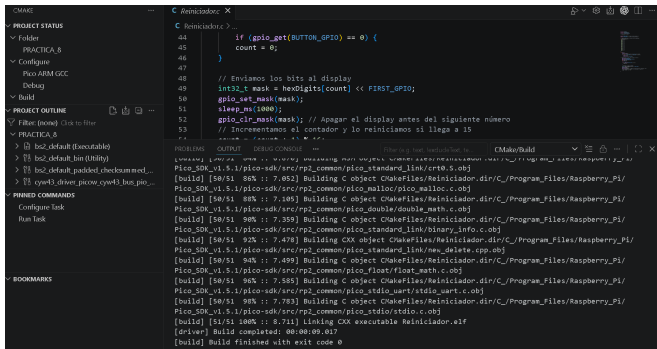


Figura 2: Código perfectamente cargado: display de 7 segmentos (cátodo común), resistencia de 220 Ω y pulsador con *pull-up* interno en GPIO 9.

resistencia externa.

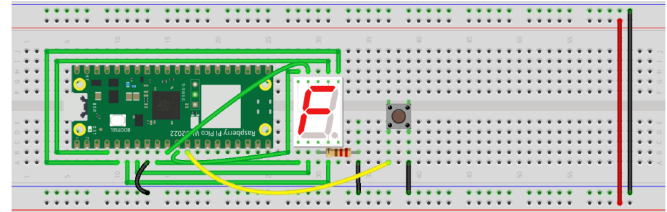


Figura 3: Diagrama de conexiones en protoboard: display de 7 segmentos (cátodo común), resistencia de 220 Ω y pulsador con *pull-up* interno en GPIO 9.

II-E Análisis del Funcionamiento del Código

El lazo principal `while(true)` ejecuta la siguiente secuencia en cada iteración:

1. Verifica si el botón está presionado (`gpio_get == 0` debido al uso de *pull-up* interno). De ser así, reinicia `count` a 0.
2. Calcula la máscara desplazando el patrón del dígito actual 2 posiciones a la izquierda para alinearlo con GPIO 2.
3. Enciende todos los segmentos del dígito en un solo ciclo de bus mediante `gpio_set_mask()`.
4. Espera 1s con `sleep_ms(1000)`.
5. Apaga todos los segmentos con `gpio_clr_mask()` antes de avanzar.
6. Incrementa el contador de forma cíclica usando el operador módulo `% 16`.

II-F Conexión del Circuito

El display de 7 segmentos de cátodo común se conectó con sus pines de segmento (A–G) a los pines GPIO 2–8 de la Pico W, cada uno en serie con una resistencia de 220 Ω para proteger los LEDs internos. El cátodo común se conectó a GND. El pulsador se conectó entre GPIO 9 y GND, aprovechando la resistencia *pull-up* interna habilitada por software, sin necesidad de

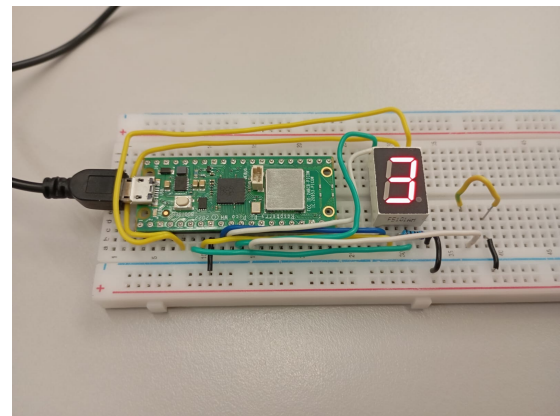


Figura 4: Circuito físico en funcionamiento mostrando un dígito en el display de 7 segmentos.

III. PREGUNTAS DE REPASO

III-A Equivalencia binaria de `hexDigits[]` y su relación con el display

La Tabla II muestra la equivalencia binaria de cada valor almacenado en el arreglo `hexDigits[]`. Cada bit corresponde a un segmento del display (bit 0 = A, bit 1 = B, ..., bit 6 = G): un bit en 1 indica que ese segmento debe encenderse para formar el dígito, y un bit en 0 indica que debe apagarse.

Tabla II: Equivalencia hexadecimal-binaria del arreglo `hexDigits[]` y segmentos activos

Dígito	Hex	Binario (bit6 G -> bit0 A)							Segmentos ON
		G	F	E	D	C	B	A	
0	0x3F	0	1	1	1	1	1	1	A,B,C,D,E,F
1	0x06	0	0	0	0	1	1	0	B,C
2	0x5B	1	0	1	1	0	1	1	A,B,D,E,G
3	0x4F	1	0	0	1	1	1	1	A,B,C,D,G
4	0x66	1	1	0	0	1	1	0	B,C,F,G
5	0x6D	1	1	0	1	1	0	1	A,C,D,F,G
6	0x7D	1	1	1	1	1	0	1	A,C,D,E,F,G
7	0x07	0	0	0	0	1	1	1	A,B,C
8	0x7F	1	1	1	1	1	1	1	A,B,C,D,E,F,G
9	0x6F	1	1	0	1	1	1	1	A,B,C,D,F,G
A	0x77	1	1	1	0	1	1	1	A,B,C,E,F,G
B	0x7C	1	1	1	1	1	0	0	C,D,E,F,G
C	0x39	0	1	1	1	0	0	1	A,D,E,F
D	0x5E	1	0	1	1	1	1	0	B,C,D,E,G
E	0x79	1	1	1	1	0	0	1	A,D,E,F,G
F	0x71	1	1	1	0	0	0	1	A,E,F,G

La relación es directa: cada patrón de bits es exactamente la codificación de qué segmentos deben iluminarse para que el display muestre visualmente ese dígito. Por ejemplo, el dígito 0 enciende todos los segmentos excepto el G (el central), lo que forma el contorno rectangular del cero; el 1 enciende únicamente B y C, los dos segmentos verticales del lado derecho.

III-B Explicación de `int32_t mask = hexDigits[count] FIRST_GPIO`

Esta línea realiza dos operaciones encadenadas:

1. **Indexación en la tabla:** `hexDigits[count]` recupera el patrón de 7 bits correspondiente al dígito actual del contador. Por ejemplo, si `count = 2`, obtiene `0x5B = 0b1011011`.
 2. **Desplazamiento a la izquierda:** el operador «`FIRST_GPIO` (es decir, " 2) desplaza ese patrón dos posiciones hacia la izquierda, de modo que el bit 0 del patrón queda alineado con el bit 2 del registro de GPIO, que corresponde a GPIO2 (segmento A). El resultado se almacena en un entero de 32 bits (`int32_t`) para garantizar que no haya desbordamiento durante el desplazamiento.
- »
- »El resultado es una máscara en la que cada bit en 1 indica exactamente cuál pin GPIO debe activarse. Esta

máscara puede pasarse directamente a `gpio_set_mask()`, que opera sobre el registro de pines del microcontrolador actualizando todos los pines indicados en un único acceso al hardware.

»III-C ¿Cuál es el rol de `gpio_set_mask()` en el programa?

»La función `gpio_set_mask(uint32_t mask)` establece en nivel alto (1 lógico, 3.3V) todos los pines GPIO cuyos bits correspondientes estén en 1 dentro de la máscara, en una sola operación atómica sobre el registro de salida del microcontrolador. Su contraparte `gpio_clr_mask()` hace lo inverso, poniendo en nivel bajo los pines indicados.

»En este programa, `gpio_set_mask()` reemplaza siete llamadas individuales a `gpio_put()`, con las siguientes ventajas: los 7 segmentos del display se activan simultáneamente (sin desfase temporal visible), el código es más conciso y eficiente, y se reduce la carga del procesador al evitar múltiples accesos independientes al periférico GPIO.

»III-D ¿Por qué no se usa una resistencia externa en el botón?

»El pulsador no requiere resistencia externa porque el programa activa la resistencia *pull-up* interna del microcontrolador mediante la función `gpio_pull_up(BUTTON_GPIO)`. Esta resistencia interna, de valor típico entre 50k■ y 80k■, conecta el pin GPIO9 internamente a 3.3V, definiendo un nivel lógico alto estable cuando el botón está en reposo (abierto). Al presionar el botón, el pin se conecta directamente a GND, sobreponiendo el nivel bajo. De este modo, la resistencia de acoplamiento queda integrada en el silicio del RP2040, simplificando el circuito externo sin perder estabilidad eléctrica (se corrigió la sintaxis de la función `gpio_get`).

»IV. CONCLUSIONES

- »■ El uso de una *lookup table* (`hexDigits[]`) para almacenar los patrones de segmentos es una técnica de programación embebida altamente eficiente: elimina cálculos en tiempo de ejecución y hace el código autoexplicativo, ya que cada entrada de la tabla tiene una correspondencia visual directa con el dígito que representa en el display.
- »■ Las funciones `gpio_set_mask()` y `gpio_clr_mask()` demostraron ser herramientas poderosas para controlar periféricos de múltiples pines. Al actualizar hasta 30 GPIO en un solo ciclo de bus, garantizan que todos los segmentos del display cambien de estado de forma simultánea, evitando artefactos visuales que aparecerían con escrituras secuenciales.
- »■ La operación de desplazamiento de bits (`<<`) es fundamental para adaptar un patrón lógico a la posición física de los pines en el microcontrolador, y su comprensión es transferible a cualquier tarea de control de hardware a bajo nivel, como la comunicación con registros de desplazamiento o buses paralelos.
- »■ La resistencia *pull-up* interna del RP2040 simplificó significativamente el circuito físico, confirmando el patrón observado en prácticas anteriores: el SDK de Raspberry Pi provee abstracciones de hardware que reducen la complejidad del diseño electrónico sin comprometer la funcionalidad.
- »■ Como mejora futura, se podría agregar un mecanismo de anti-rebote al botón de reinicio, ya que en la implementación actual una pulsación con rebote podría generar múltiples reinicios consecutivos. También sería posible extender el proyecto a un display de doble dígito para

contar en decimal del 0 al 99.

»

»ANEXOS

»A. Código fuente – Reiniciador.c

»ANEXO A – Reiniciador.c (completo)

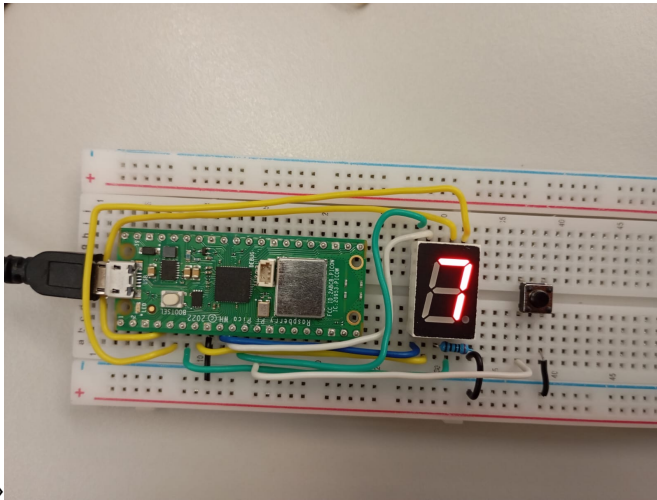
```
»#include <stdio.h>
»#include "pico/stdlib.h"
»#include "hardware/gpio.h"
»
»#define FIRST_GPIO 2
»#define BUTTON_GPIO 9
»
»int hexDigits[16] = {
»    0x3F, 0x06, 0x5B, 0x4F,
»    0x66, 0x6D, 0x7D, 0x07,
»    0x7F, 0x6F, 0x77, 0x7C,
»    0x39, 0x5E, 0x79, 0x71
»};
»
»int main() {
»    stdio_init_all();
»
»    for (int i = FIRST_GPIO; i <
»        FIRST_GPIO + 7; i++) {
»        gpio_init(i);
»        gpio_set_dir(i, GPIO_OUT);
»    }
»
»    gpio_init(BUTTON_GPIO);
»    gpio_set_dir(BUTTON_GPIO,
»        GPIO_IN);
»    gpio_pull_up(BUTTON_GPIO);
»
»    int count = 0;
»
»    while (true) {
»        if (gpio_get(BUTTON_GPIO)
»            == 0) {
»            count = 0;
»        }
»        int32_t mask = hexDigits[
»            count] << FIRST_GPIO;
»        gpio_set_mask(mask);
»        sleep_ms(1000);
»        gpio_clr_mask(mask);
»        count = (count + 1) % 16;
»    }
»}
```


»B. Archivo CMakeLists.txt

```
»ANEXO B – CMakeLists.txt (completo)

»cmake_minimum_required(VERSION
    3.13)
»
»include(pico_sdk_import.cmake)
»set(PICO_BOARD pico_w)
»
»project(PRACTICA_8)
»
»pico_sdk_init()
»
»add_executable(Reiniciador
    Reiniciador.c
)
»
»target_link_libraries(Reiniciador
    pico_stdlib
)
»
»pico_add_extra_outputs(Reiniciador
)
»
```

»C. Diagrama en Protoboard



»Figura 5: Diagrama funcional del protoboard.